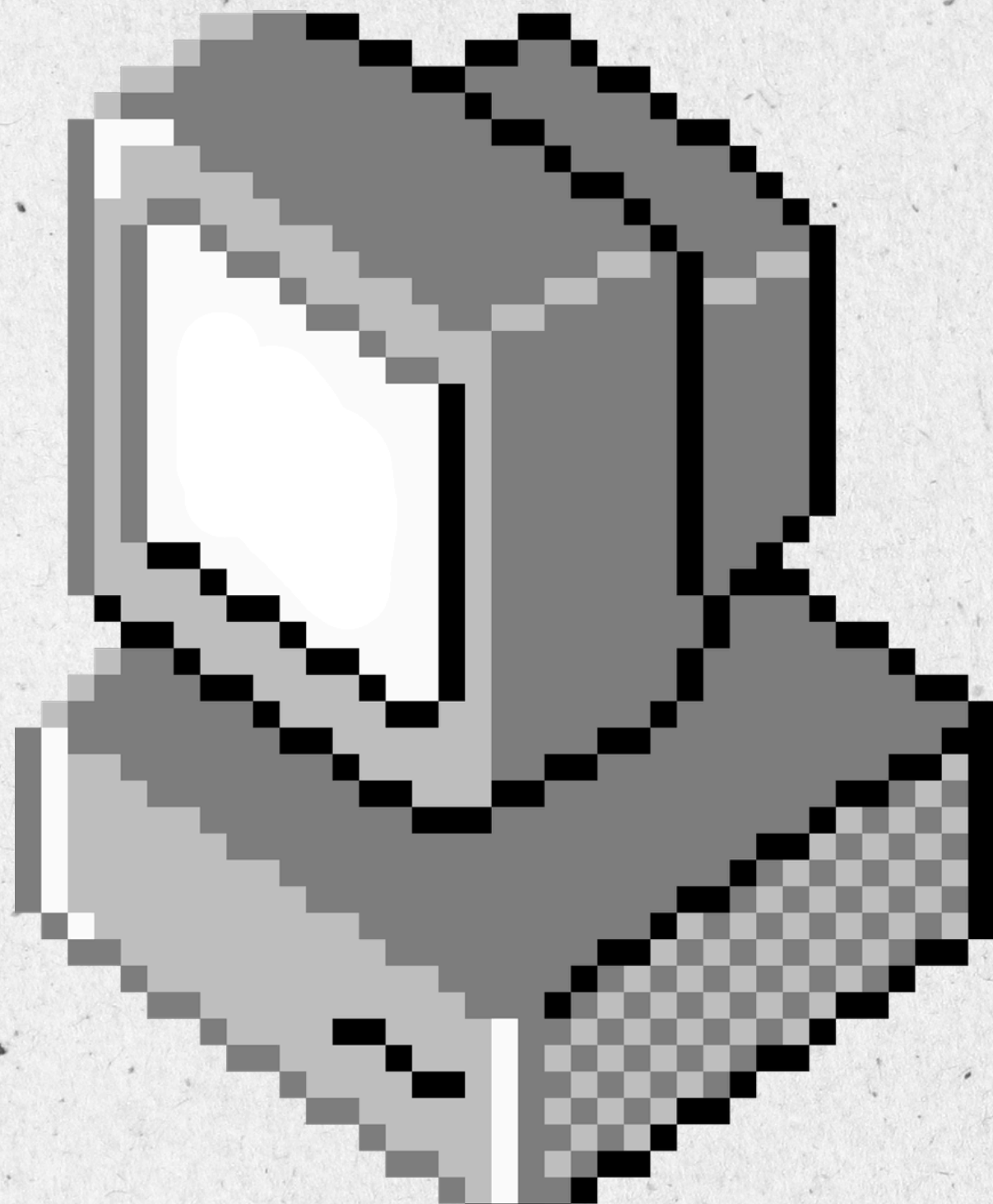
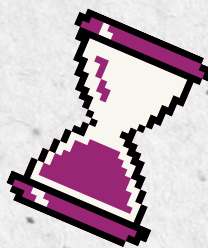
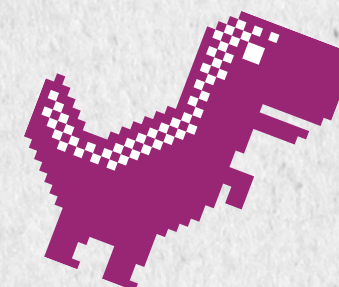




SOLUÇÃO DO PROBLEMA DO CAIXEIRO VIAJANTE (TSP) EUCLIDIANO

Henrique Reis, Luísa Böhm, Murilo Salem



PROBLEMA

O Problema do Caixeiro Viajante (TSP) é um dos desafios matemáticos e computacionais mais famosos e estudados da história.

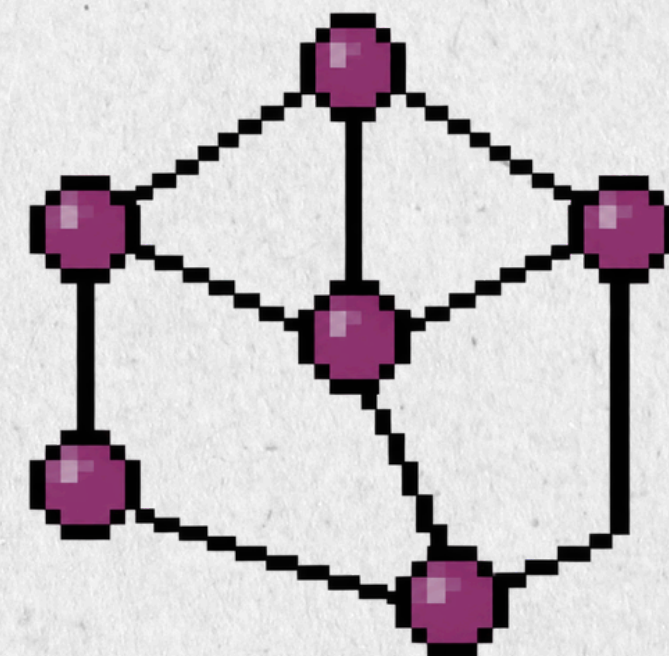
Em termos simples, o problema consiste em: dado uma lista de cidades e as distâncias entre cada par delas, qual é a rota mais curta possível que visita cada cidade exatamente uma vez e retorna à cidade de origem?

SURGIMENTO

O conceito geral já aparecia em manuais práticos para caixeiros viajantes na Alemanha do século XIX.

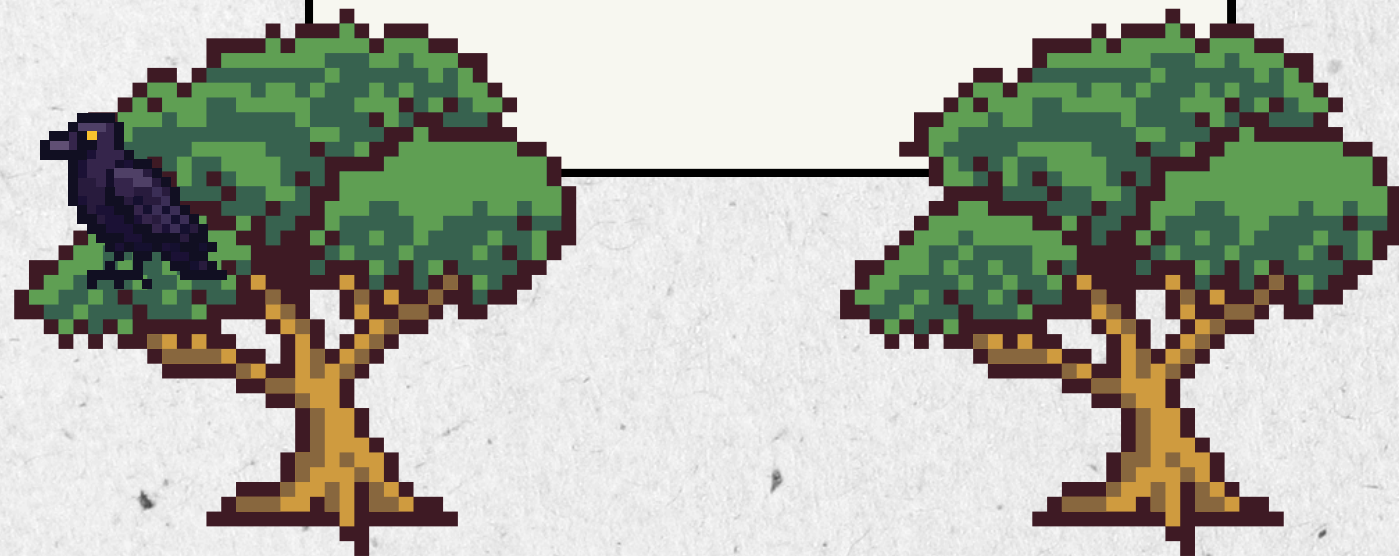
LIMITE COMPUTACIONAL

Encontrar a solução absolutamente perfeita (ótima) por meio da "força bruta" (testando todas as combinações) torna-se impossível até para os supercomputadores mais rápidos do mundo quando o número de cidades é moderadamente alto.



ALGORITMOS

Branch and
Bound



Vizinho mais
Próximo



ALGORITMO EXATO – BRANCH AND BOUND

É um método exato de otimização usado para resolver problemas complexos e combinatórios, como o problema da mochila e o caixeiro-viajante.

Organiza a busca pela solução ótima dividindo o problema original em partes menores:

- **Branch (Ramificação):** Divide o espaço de soluções em subproblemas, criando uma árvore de escolhas. Cada nó representa um caminho ou estado.
- **Bound (Limitação):** Calcula limites (custos mínimos ou máximos) para cada subproblema.
- **Poda (Pruning):** Descarta ramos inteiros da árvore se o limite provar que o caminho não pode gerar uma solução melhor do que a já encontrada.

ALGORITMO APROXIMATIVO - VIZINHO MAIS PRÓXIMO



O método do vizinho mais próximo é um algoritmo baseado na proximidade de dados para realizar classificações ou previsões.

Funcionamento:

- **Conceito básico:** O algoritmo assume que pontos de dados semelhantes ficam próximos uns dos outros no espaço.
- **Cálculo de distância:** Ele mede a distância entre um novo ponto e os pontos já conhecidos no conjunto de dados.
- **Definição de k:** O parâmetro k define quantos vizinhos mais próximos serão consultados para tomar a decisão.
- **Classificação ou Regressão:** Em classificação, o novo ponto ganha a classe majoritária entre os k vizinhos; em regressão, calcula-se a média dos valores desses vizinhos.

PROTOCOLO EXPERIMENTAL

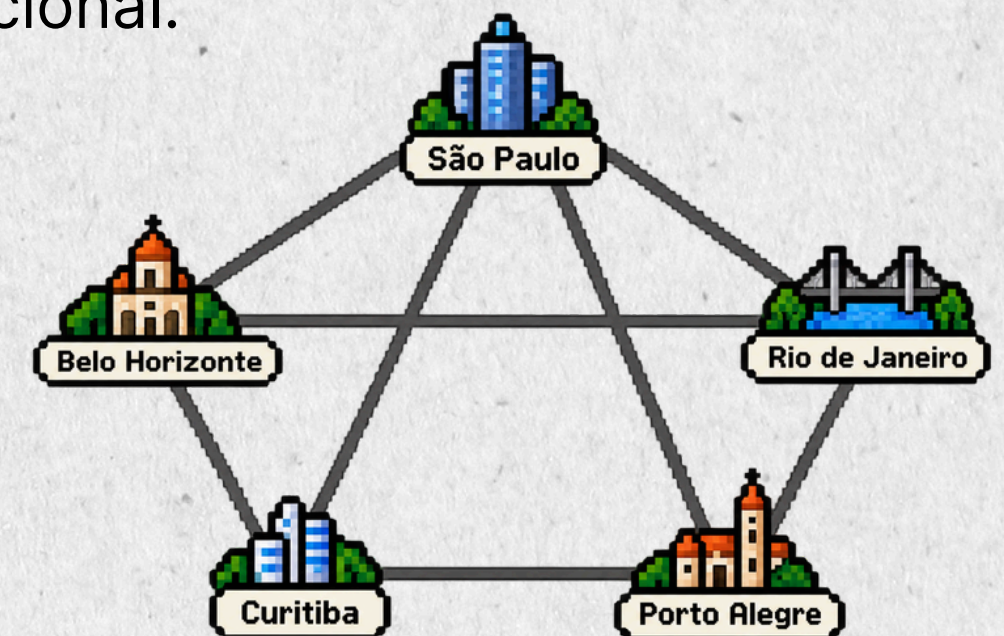
Instâncias de Teste: Foram utilizadas cinco instâncias com matrizes de adjacência de dimensões crescentes:

- tsp1_253.txt (11 cidades)
- tsp2_1248.txt (6 cidades)
- tsp3_1194.txt (15 cidades)
- tsp4_7013.txt (44 cidades)
- tsp5_27603.txt (29 cidades)

Repetições e Confiabilidade: Cada instância foi executada independentemente por ? iterações consecutivas para mitigar oscilações provocadas por processos concorrentes do sistema operacional.

Métricas Avaliadas:

- Qualidade da Solução
- Diferença Relativa (Gap Percentual)
- Tempo de Execução
- Conclusão/Timeout (apenas para o exato)



ALGORITMO EXATO

Representação dos Estados:

- Cada nó na árvore de busca representa um caminho parcial percorrido até determinado momento.
- O nó armazena a lista de arestas percorridas, o vetor booleano de cidades visitadas, o custo acumulado real, a cidade atual, o nível de profundidade na árvore e o limite inferior estimado.

Estratégia de Exploração:

- É utilizada uma `std::priority_queue` parametrizada com o operador `greater<Node>` para operar como uma min-heap com base no atributo `bound`.

Cálculo do Limite Inferior (Lower Bound):

- Para cada vértice, o algoritmo determina a aresta de menor peso (`firstMin`) e a segunda menor aresta incidente (`secondMin`).

ALGORITMO EXATO

Critério de Poda (Pruning):

- Mantém-se uma variável com a melhor solução completa já encontrada (`final_res`), inicializada com `INT_MAX`.
- Verifica-se se `curr.bound >= final_res`. Caso positivo, o nó é descartado imediatamente, podando todo o sub-ramo de busca associado.

Fecho do Ciclo e Atualização da Solução Ótima:

- Ao atingir a profundidade máxima, o algoritmo conecta a última cidade de volta à cidade inicial (vértice 0).
- Se o custo total do ciclo for estritamente menor que `final_res`, a variável `final_res` e o caminho ótimo são atualizados com a nova melhor solução.

Controle de Tempo (Timeout):

- Se o tempo de execução ultrapassar o limiar de 60 segundos, a busca é interrompida para prevenir o bloqueio indefinido decorrente do crescimento fatorial da árvore de busca nas instâncias maiores.

ALGORITMO APROXIMATIVO

Inicialização e Estruturas de Dados:

- O algoritmo aloca um vetor booleano visited inicializado com false para controlar os vértices já inseridos na rota.
- Define-se deterministicamente a cidade 0 como o ponto de partida, marcando $\text{visited}[0] = \text{true}$.

Passo Guloso (Greedy Choice):

- Em cada iteração, percorre todas as cidades do grafo e avalia as distâncias a partir de current_city na matriz de adjacência.
- Identifica a cidade não visitada que apresenta o menor custo de deslocação.

Transição de Estado:

- A cidade identificada é marcada como visitada e inserida no vetor path.
- O custo da aresta percorrida é somado à variável acumuladora.

Fecho do Ciclo Hamiltoniano:

- O vértice inicial 0 é anexado ao final do caminho percorrido no vetor path, completando o circuito fechado e totalizando o custo final aproximado.



RESULTADOS

			Branch and Bound			Vizinho mais próximo		
txt	idades	c. ótimo	tempo	c. obtido	gap	tempo	c. obtido	gap
tps1	11	253	0,172889	254	0,395%	0,000007	299	18,1818%
tps2	6	1248	0,000010	1248	0,00000%	0,000005	1272	1,9231%
tps3	15	1194	0,019320	1194	0,00000%	0,000006	1260	5,5276%
tps4	44	7013	timeout	-	-	0,000008	10587	50,9625%
tps5	29	27603	timeout	-	-	0,000007	36399	31,8661%

CONCLUSÕES

- Compromisso entre Eficiência Temporal e Qualidade da Solução (Trade-off).
- Impacto da Explosão Combinatória no Método Exato.
- Degradação e Miopia da Heurística Gulosa.
- Divergência na Instância tsp1_253.txt.

